



UNIVERSIDADE FEEVALE

Sistemas Operacionais

TRABALHO FINAL DE SISTEMAS OPERACIONAIS

**Mateus Volkart
Natália Reis Granetto
Haniel Nasiniak
Pedro de Souza Ruduit**

2026

SUMÁRIO

1 INTRODUÇÃO.....	2
2 DESCRIÇÃO DO ALGORITMO.....	2
3 PREMISSAS.....	3
4 ESTRUTURA DE DADOS.....	3
5 FLUXO DE EXECUÇÃO.....	5
6 EXECUÇÃO.....	6
7 SAÍDA DO SIMULADOR.....	6
8 CONCLUSÃO.....	7

1 INTRODUÇÃO

O objetivo deste trabalho é desenvolver um simulador de escalonamento de processos, abordando na prática como um Sistema Operacional gerencia o uso da CPU e o compartilhamento de recursos entre múltiplos programas. O simulador construído representa, de forma simplificada, a evolução temporal e o ciclo de vida dos processos em um sistema computacional, acompanhando sua transição entre os estados de novo, pronto, em execução, bloqueado por I/O e finalizado.

O escalonamento de processos é uma das principais tarefas de um Sistema Operacional, pois permite organizar a execução simultânea de diversos programas, garantindo um melhor aproveitamento dos recursos disponíveis e um compartilhamento eficiente da CPU entre os processos.

Para a realização deste projeto, foi implementado o algoritmo *Round Robin com Feedback*, aplicando múltiplas filas de prioridade e tratamento de operações de entrada e saída. Além disso, o projeto também foi configurado para ser executado tanto pelo interpretador Python quanto por meio do Docker, possibilitando maior portabilidade e facilidade de execução em diferentes ambientes.

2 DESCRIÇÃO DO ALGORITMO

A implementação do algoritmo foi feita usando um laço principal que funciona como o relógio do sistema, avançando uma unidade de tempo por vez (de 1 em 1). Esse laço controla o estado e a prioridade de cada processo usando duas filas: alta prioridade e baixa prioridade.

O escalonador sempre prioriza a fila de alta prioridade, executando os processos da fila de baixa prioridade apenas quando a primeira estiver vazia. Quando um processo é selecionado para entrar na CPU, ele pode executar por, no máximo, três unidades de tempo, que é o limite do quantum.

Caso o processo finalize a execução ou solicite uma entrada e saída (I/O) antes do término do quantum, a CPU é liberada imediatamente. Se o processo usar as três unidades do quantum, ele sofre preempção (é interrompido), retorna seu status para PRONTO e é inserido no final da fila de baixa prioridade. Os processos

bloqueados permanecem na fila de I/O, onde seu tempo de espera é atualizado a cada avanço do relógio do sistema.

Enquanto tudo isso acontece na CPU, os processos que estão bloqueados ficam guardados em uma fila de I/O esperando a finalização de suas operações. Quando o I/O de um processo termina, ele volta para as filas dependendo do dispositivo que usou: se foi o disco, ele vai para o final da fila de baixa prioridade, se foi a fita magnética ou a impressora, ele volta direto para o final da fila de alta prioridade.

3 PREMISSAS

Para que o simulador efetue de forma controlada e de acordo com os requisitos propostos, foram definidas algumas premissas e parâmetros fixos no início do código. A quantidade de processos foi definida em 5, e todos eles são criados simultaneamente no instante inicial da simulação ($T=0$). O tempo total de CPU que cada um precisa para terminar é gerado de forma aleatória, variando entre 8 e 20 unidades de tempo. O momento em que a solicitação de I/O acontecerá durante a execução do processo é determinado de forma aleatória no instante da sua criação.

O quantum do escalonador foi fixado em 3 unidades de tempo. Nas operações de entrada e saída (I/O), o tempo de bloqueio também é aleatório, variando entre 2 e 5 unidades de tempo, independentemente do dispositivo utilizado (Disco, Fita Magnética ou Impressora). Por fim, para que os testes sejam sempre iguais, foi utilizada uma semente aleatória (seed) fixa, definida pelo valor 4867654453.

4 ESTRUTURA DE DADOS

O PCB (Bloco de Controle de Processo), que guarda os dados de cada processo, foi representado por meio de um dicionário do Python, no qual são armazenadas informações como o PID, o tempo restante da CPU, o momento do I/O, o tipo de dispositivo utilizado e o status atual do processo.

```
def criar_processo(pid):
    tempo_total_cpu = random.randint(8, 20)

    tem_io = random.choice([True, False])

    if tem_io:
        tipo_io = random.choice(["DISCO", "FITA", "IMPRESSORA"])
        tempo_total_io = random.randint(2, 5)
        momento_io = random.randint(max(1, tempo_total_cpu // 3), max(2, tempo_total_cpu - 2))
        realizou_io = False
    else:
        tipo_io = "NENHUM"
        tempo_total_io = 0
        momento_io = -1
        realizou_io = True

    return {
        "pid": pid,
        "ppid": 0,
        "status": "NOVO",
        "prioridade": "ALTA",
        "tempo_cpu_restante": tempo_total_cpu,
        "tipo_io": tipo_io,
        "tempo_total_io": tempo_total_io,
        "tempo_io_restante": 0,
        "realizou_io": realizou_io,
        "momento_io": momento_io
    }
```

Fonte: Autoria própria

As duas filas de alta e baixa prioridade foram implementadas utilizando a estrutura deque da biblioteca Collections do Python. Essa estrutura foi escolhida por apresentar um comportamento adequado ao modelo FIFO (First In, First Out), permitindo inserir elementos no final da fila e removê-los do início de forma eficiente. Já a fila de I/O foi implementada utilizando uma lista comum do Python, facilitando o percurso pelos processos bloqueados para atualizar seus tempos de espera a cada avanço do relógio do sistema.

```
13  fila_alta_prioridade = deque()
14  fila_baixa_prioridade = deque()
15  fila_io = []
```

Fonte: Autoria própria

5 FLUXO DE EXECUÇÃO

Para demonstrar o funcionamento do simulador ao longo do tempo, nós organizamos o fluxo de execução mostrando o ciclo de vida e a mudança de estado dos processos. Tudo começa na etapa de criação, onde optamos por fazer com que todos os processos comecem juntos logo no início da simulação, no tempo zero ($T=0$). Escolhemos essa abordagem porque ficou mais simples de controlar e acompanhar o comportamento do sistema desde o ponto de partida. Assim que são criados com o status NOVO, a semente aleatória calcula o tempo total de CPU que cada um vai precisar e o momento em que vão pedir uma entrada e saída. Logo em seguida, todos mudam para o status PRONTO e entram na fila de alta prioridade.

Na hora da execução, a nossa principal escolha de lógica foi fazer o relógio geral do sistema avançar estritamente de 1 em 1 ciclo de tempo. Nós decidimos fazer o código rodar passo a passo porque, enquanto um processo está com o status EXECUTANDO no processador, a fila de I/O precisa continuar diminuindo o tempo dos outros processos em segundo plano. Se o simulador avançasse o tempo do quantum de uma vez só, os processos que estivessem esperando um dispositivo iam ficar congelados no tempo. Portanto, atualizar o relógio de 1 em 1 segundo foi a solução que o grupo encontrou para simular o paralelismo do computador de forma simples.

O destino do processo depois de passar pela CPU depende do seu próprio tempo de execução. Se o tempo restante de CPU dele chegar a zero, ocorre a finalização: o processo muda para o status FINALIZADO e sai do circuito do simulador. Caso o processo atinja o momento planejado para fazer uma entrada e saída antes do fim do seu turno, acontece o bloqueio: ele muda para BLOQUEADO e vai para a fila de I/O. Por outro lado, se o processo não pedir I/O e tentar usar as 3 unidades do quantum inteiras sem terminar, o sistema operacional força a preempção. Quando isso acontece, o processo volta para o status PRONTO, e vai para o final da fila de baixa prioridade, evitando que um processo muito longo segure a CPU sozinho e deixe os outros esperando por tempo indeterminado. O ciclo termina quando o tempo de I/O acaba e o processo volta a ficar PRONTO, passando pelas regras de feedback para decidir sua nova fila.

6 EXECUÇÃO

Para executar o nosso projeto, existem duas maneiras diferentes. A primeira forma é a tradicional, direto pelo terminal da máquina, usando o próprio interpretador do Python. Para isso, basta abrir o terminal na pasta raiz do projeto e digitar o comando:

```
python src/escalador.py
```

A segunda forma é usando o Docker, que foi a nossa atividade bônus para deixar o ambiente isolado e fácil de testar. Para isso, criamos um arquivo Dockerfile que usa uma imagem leve oficial do Python (python:3.11-slim), cria uma pasta de trabalho interna chamada /app, copia os nossos códigos da pasta src para lá e define o comando automático que vai iniciar o script. Para juntar tudo e orquestrar a execução, também criamos o arquivo docker-compose.yml. Com isso pronto, para rodar o simulador pelo Docker sem precisar configurar nada na máquina, basta abrir o terminal na raiz e usar o comando:

```
docker compose up
```

7 SAÍDA DO SIMULADOR

A saída do simulador foi feita para apresentar, passo a passo, os eventos que acontecem no sistema direto na tela do terminal. Cada linha do log começa com um marcador de tempo, como [t=1], [t=2], e assim por diante. Esse número é o relógio geral da simulação. Isso permite acompanhar exatamente em qual momento um processo começou a rodar, quando ele foi preemptado, quando ficou bloqueado esperando um dispositivo ou quando foi finalizado. No final de toda a execução, o programa exibe um relatório de resumo com as estatísticas finais, quantos processos foram rodados/finalizados e também o tempo total que levou para terminar todos os processos.

8 CONCLUSÃO

Desenvolver esse simulador foi importante para entender, na prática, como um Sistema Operacional distribui a CPU entre diversos programas executados ao mesmo tempo. Embora a teoria estudada em sala de aula pareça simples, a implantação mostrou que o controle das prioridades e a alternância entre processos exigem uma grande lógica e planejamento.

A maior dificuldade encontrada foi sincronizar o relógio global da simulação: fazer o tempo do quantum rodar na CPU e, ao mesmo tempo, diminuir o tempo de espera dos processos que estavam travados na fila de I/O, sem interromper o funcionamento das demais partes do sistema.